

# CSE 4621: Machine Learning

# Convolutional Neural Networks

## Lecture 8

Ishmam Tashdeed  
Lecturer, CSE IUT

# Lecture Outline

- Images as Data
  - What is an image
  - Common computer vision tasks
  - Why MLPs struggle with images
- Convolution Operation
  - Inductive biases
  - Convolution vs. cross-correlation
- Building Blocks
  - Pooling layers
  - Architecture design choices: depth, width, stride, padding
- Classic Architectures
- Beyond Training From Scratch
  - Transfer learning & fine-tuning

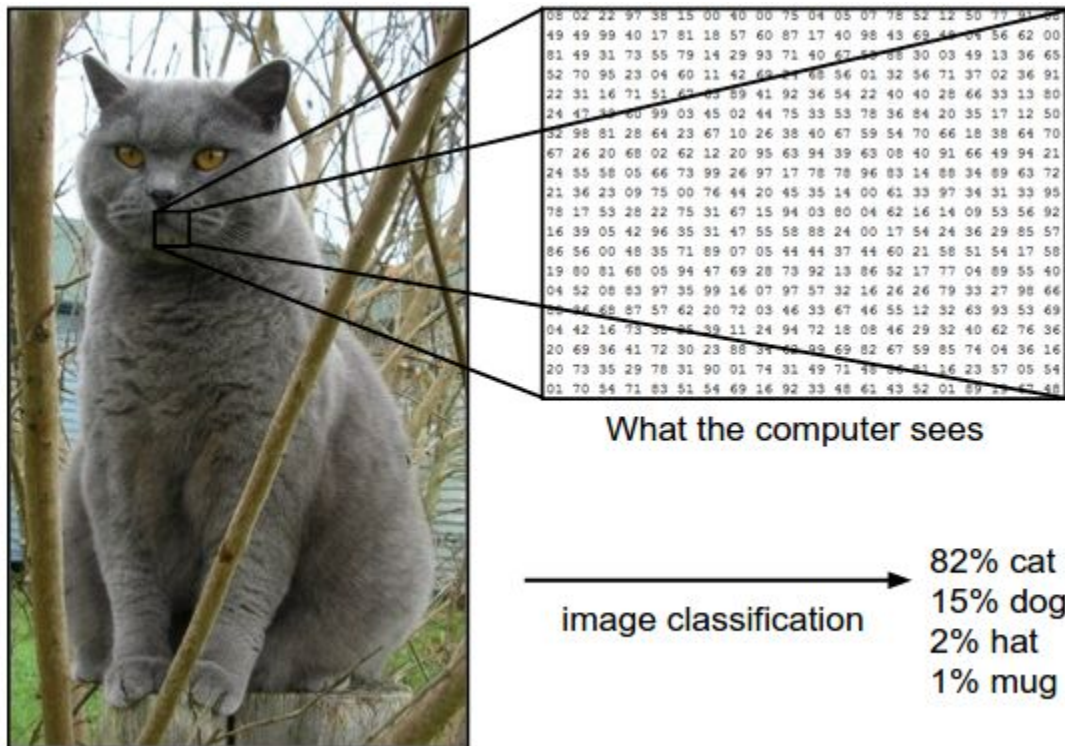
# What Is an Image?

- An image consists of multiple pixels, each representing intensity, typically in a range of  $[0, 255]$
- In case of colored images, it contains 3 sets of pixels, each representing a single color channel (Red, Green or Blue)
- An image is a tensor of the shape:  $\mathbf{X} \in \mathbb{R}^{H \times W \times C}$

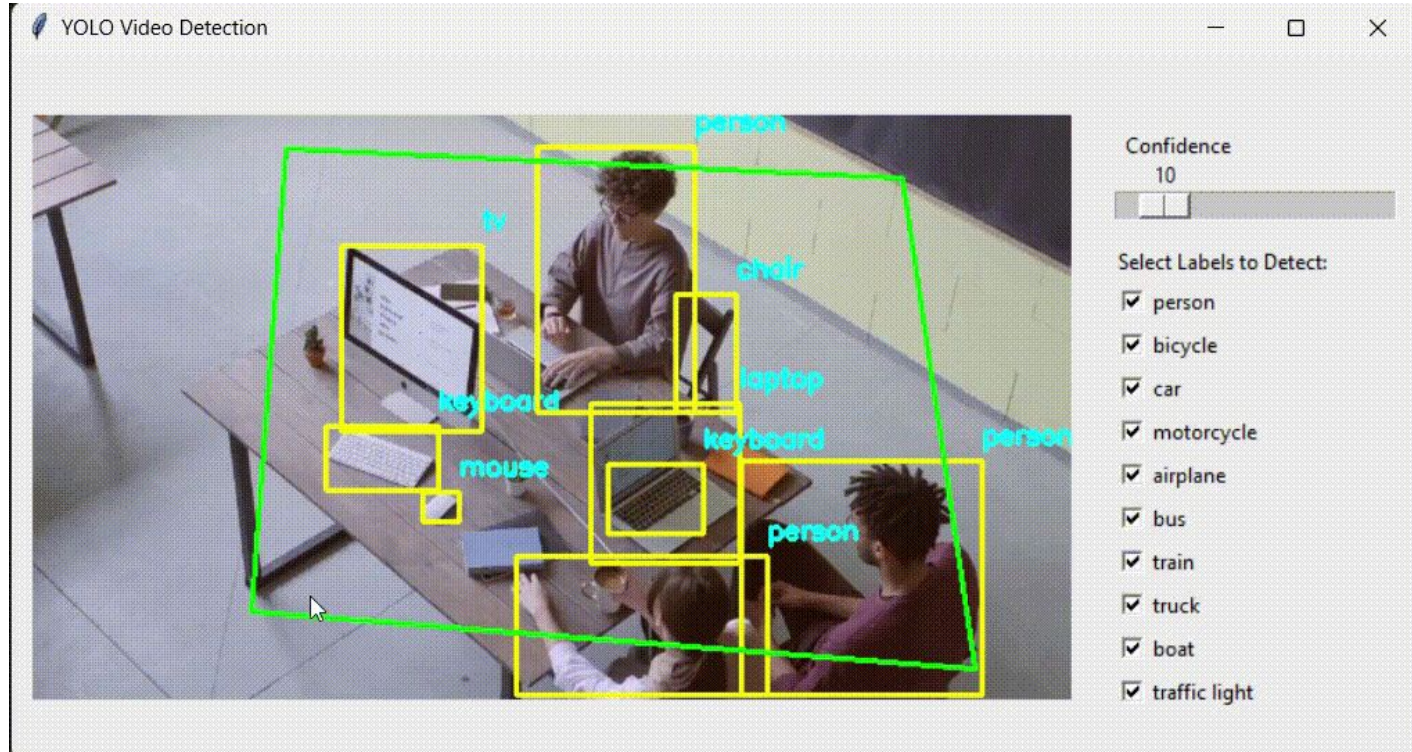


|     |     |     |    |
|-----|-----|-----|----|
| 241 | 240 | ... | 52 |
|-----|-----|-----|----|

# Computer Vision Tasks

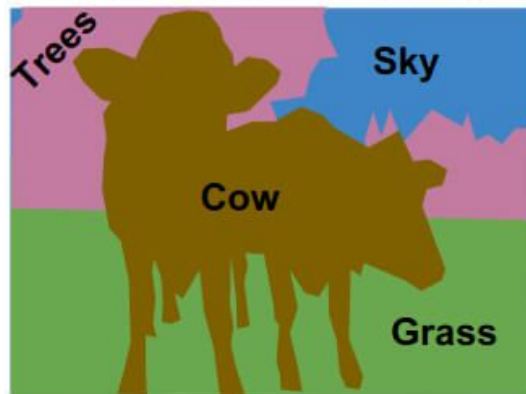
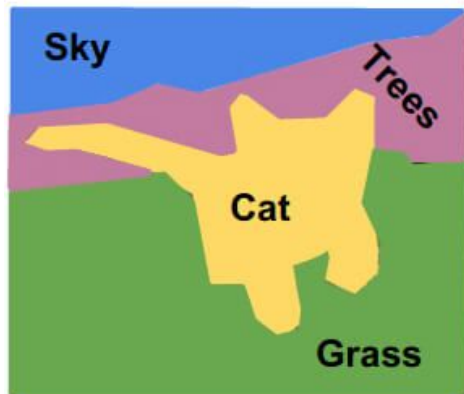


# Computer Vision Tasks

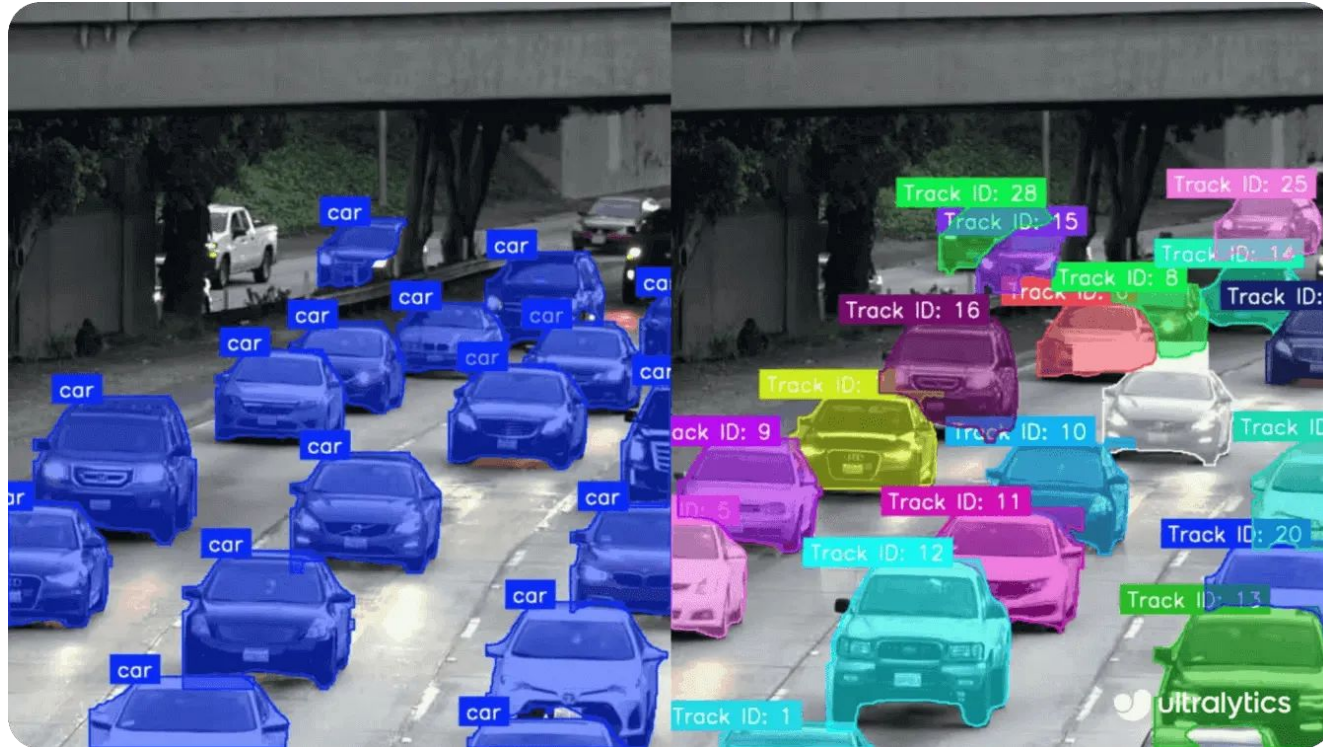




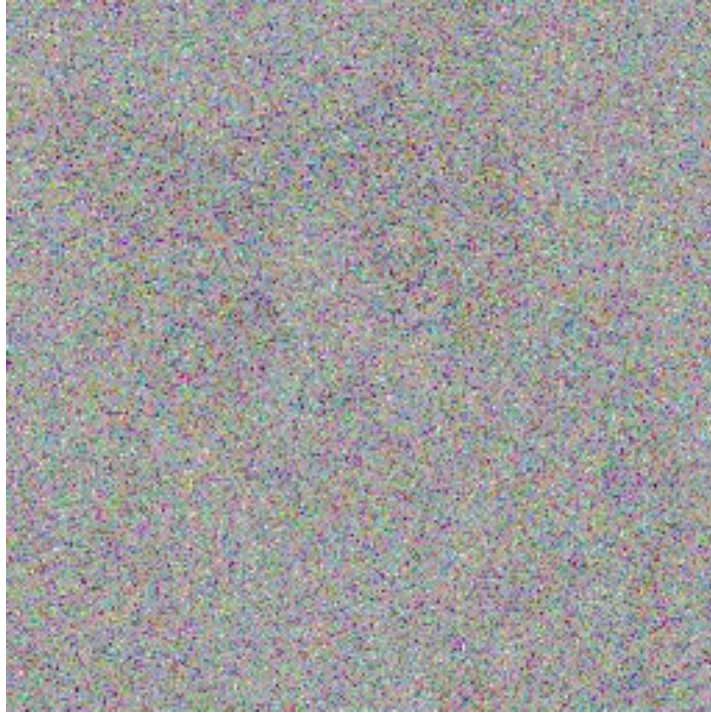
# Computer Vision Tasks



# Computer Vision Tasks



# Computer Vision Tasks





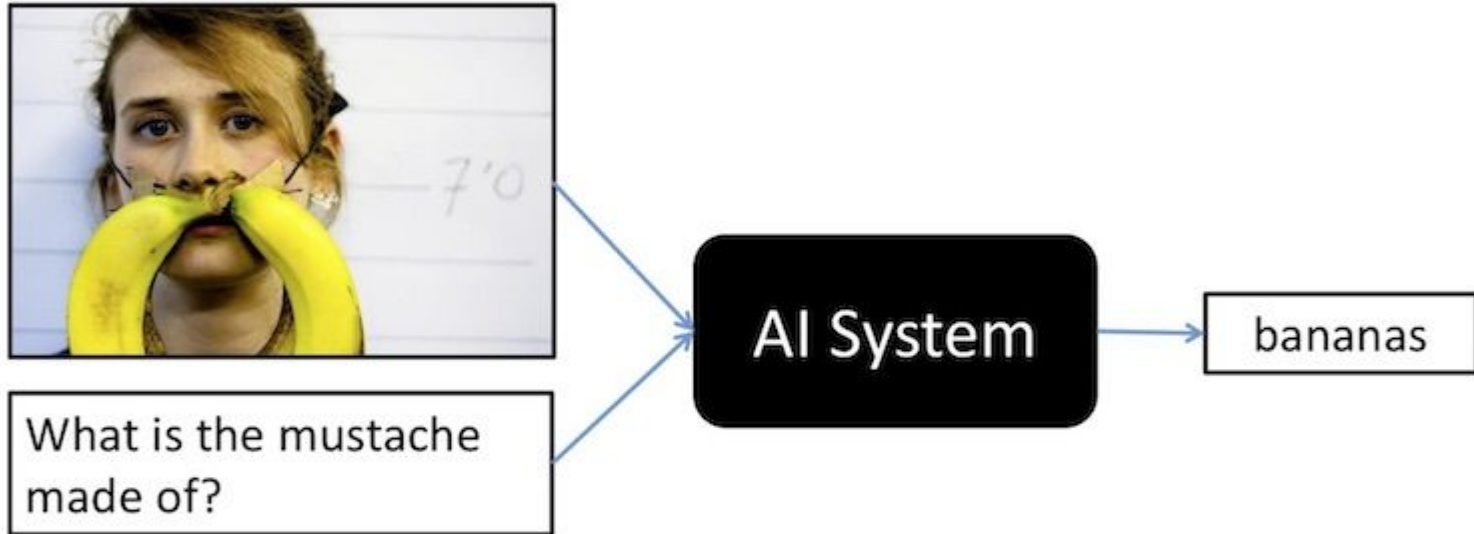
# Computer Vision Tasks



Captioning Model

*A happy dog is standing in the ocean*

# Computer Vision Tasks



# Computer Vision Tasks

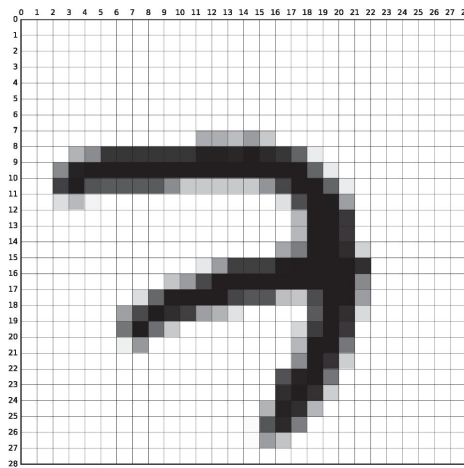
All of these tasks share a common need → *extracting meaningful visual features* (edges, textures, shapes, objects) from raw pixels

Let's try to solve a simple computer vision problem → Handwritten digit recognition

A single image consists of:

$32 \times 32 = 1024$  pixels

Let's solve this problem using an NN!

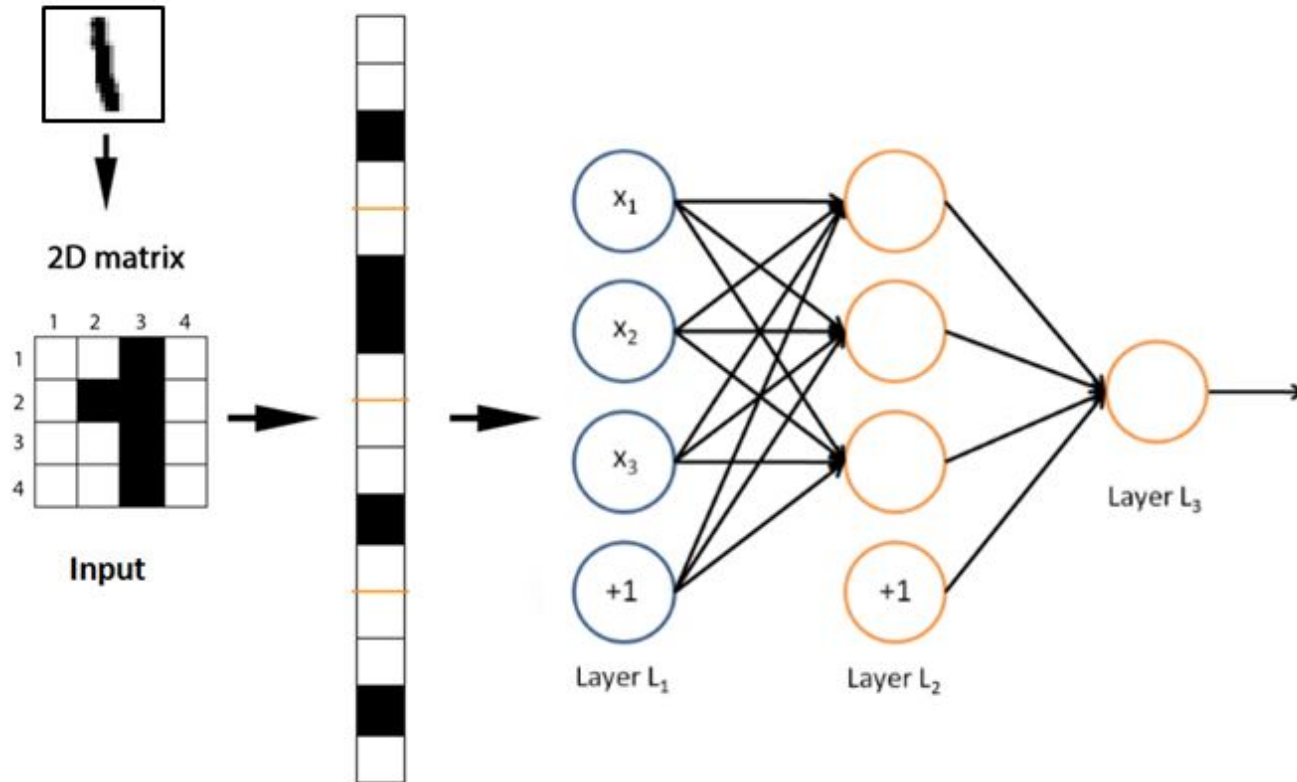


(a) MNIST sample belonging to the digit '7'.

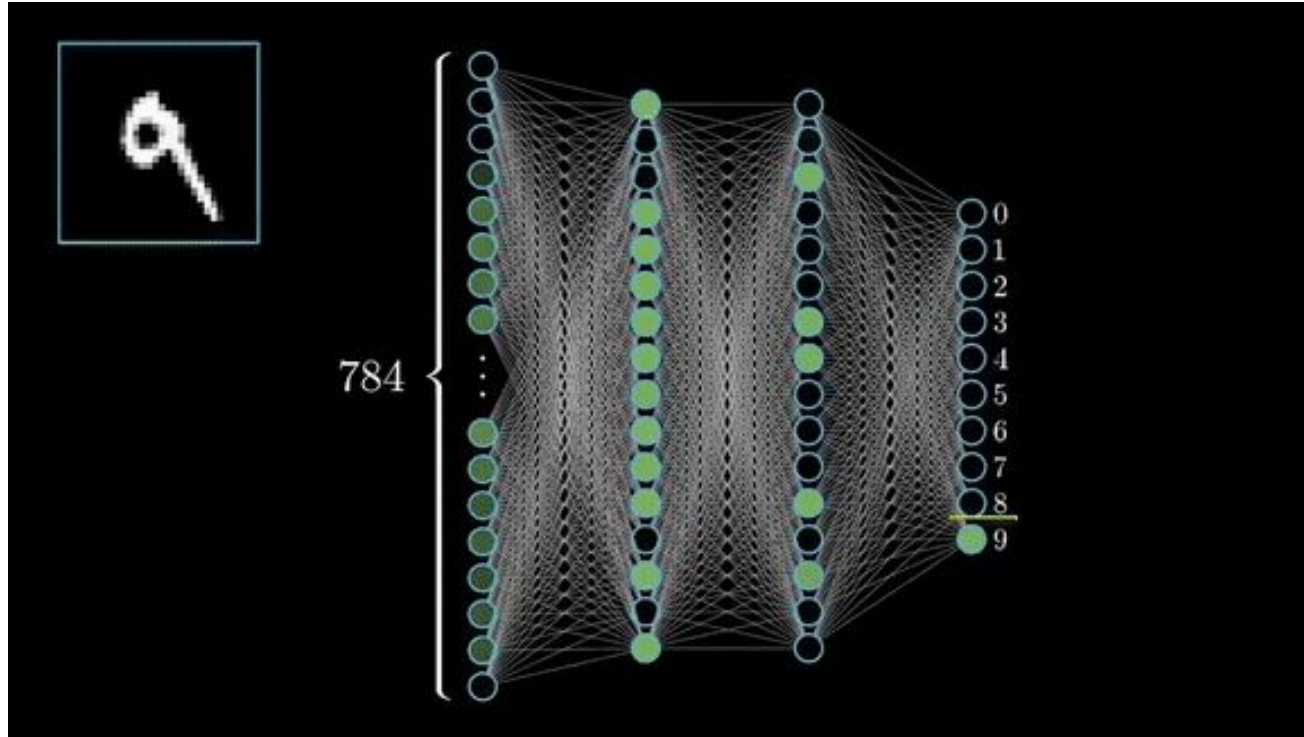


(b) 100 samples from the MNIST training set.

# Computer Vision Tasks



# Computer Vision Tasks





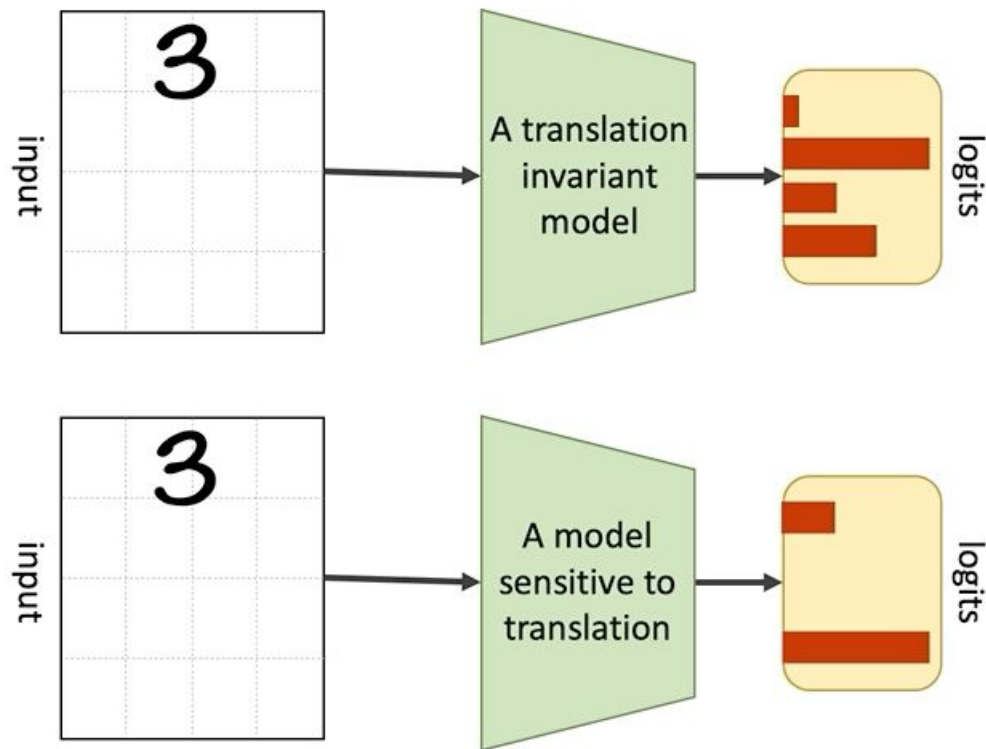
# Computer Vision Tasks

For a  $32 \times 32 \times 3$  CIFAR-10 image: input vector has 3,072 dimensions. First hidden layer with 512 units:  $3072 \times 512 \approx 1.5\text{M}$  parameters

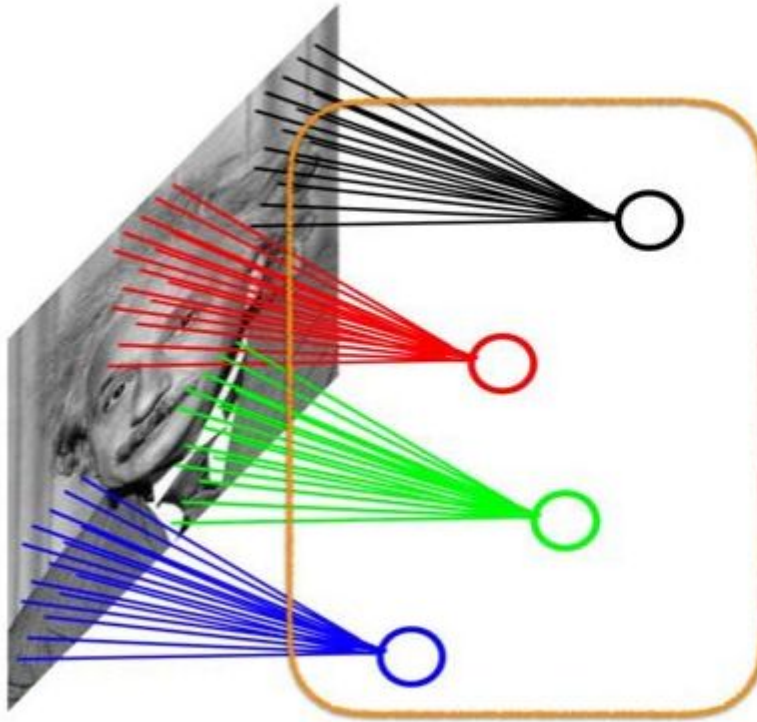
There are 3 fundamental issues:

- Parameter explosion: Far too many weights to learn efficiently; massive overfitting risk with limited data
- No spatial structure exploited: A pixel and its neighbour are treated as completely unrelated inputs — same as if they were on opposite corners of the image
- No translation invariance: The MLP has no mechanism to share this knowledge across positions

# Inductive Biases for Vision



# Locality



|    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 08 | 02 | 22 | 97 | 38 | 15 | 00 | 40 | 00 | 75 | 04 | 05 | 07 | 78 | 52 | 12 | 50 | 77 | 91 | 28 |
| 49 | 49 | 99 | 40 | 17 | 81 | 18 | 57 | 60 | 87 | 17 | 40 | 98 | 43 | 68 | 48 | 04 | 56 | 62 | 00 |
| 81 | 49 | 31 | 73 | 55 | 79 | 14 | 29 | 93 | 71 | 40 | 67 | 54 | 38 | 30 | 03 | 49 | 13 | 36 | 65 |
| 52 | 70 | 95 | 23 | 04 | 60 | 11 | 42 | 63 | 41 | 63 | 56 | 01 | 32 | 56 | 71 | 37 | 02 | 36 | 91 |
| 22 | 31 | 16 | 71 | 51 | 65 | 83 | 89 | 41 | 92 | 36 | 54 | 22 | 40 | 40 | 28 | 66 | 33 | 13 | 80 |
| 24 | 47 | 34 | 05 | 99 | 03 | 45 | 02 | 44 | 75 | 33 | 53 | 78 | 36 | 84 | 20 | 35 | 17 | 12 | 50 |
| 32 | 98 | 81 | 28 | 64 | 23 | 67 | 10 | 26 | 38 | 40 | 67 | 59 | 54 | 70 | 66 | 18 | 38 | 64 | 70 |
| 67 | 26 | 20 | 68 | 02 | 62 | 12 | 20 | 95 | 63 | 94 | 39 | 63 | 08 | 40 | 91 | 66 | 49 | 94 | 21 |
| 24 | 55 | 58 | 05 | 66 | 73 | 99 | 26 | 97 | 17 | 78 | 78 | 96 | 83 | 14 | 88 | 34 | 89 | 63 | 72 |
| 21 | 36 | 23 | 09 | 75 | 00 | 76 | 44 | 20 | 45 | 35 | 14 | 00 | 61 | 33 | 97 | 34 | 31 | 33 | 95 |
| 78 | 17 | 53 | 28 | 22 | 75 | 31 | 67 | 15 | 94 | 03 | 80 | 04 | 62 | 16 | 14 | 09 | 53 | 56 | 92 |
| 16 | 39 | 05 | 42 | 96 | 35 | 31 | 47 | 55 | 58 | 88 | 24 | 00 | 17 | 54 | 24 | 36 | 29 | 85 | 57 |
| 86 | 56 | 00 | 48 | 35 | 71 | 89 | 07 | 05 | 44 | 44 | 37 | 44 | 60 | 21 | 58 | 51 | 54 | 17 | 58 |
| 19 | 80 | 81 | 68 | 05 | 94 | 47 | 69 | 28 | 73 | 92 | 13 | 86 | 52 | 17 | 77 | 04 | 89 | 55 | 40 |
| 04 | 52 | 08 | 83 | 97 | 35 | 99 | 16 | 07 | 97 | 57 | 32 | 16 | 26 | 26 | 79 | 33 | 27 | 98 | 66 |
| 05 | 44 | 68 | 87 | 57 | 62 | 20 | 72 | 03 | 46 | 33 | 67 | 46 | 55 | 12 | 32 | 63 | 93 | 53 | 69 |
| 04 | 42 | 16 | 73 | 55 | 85 | 39 | 11 | 24 | 94 | 72 | 18 | 08 | 46 | 29 | 32 | 40 | 62 | 76 | 36 |
| 20 | 69 | 36 | 41 | 72 | 30 | 23 | 88 | 34 | 45 | 83 | 69 | 82 | 67 | 59 | 85 | 74 | 04 | 36 | 16 |
| 20 | 73 | 35 | 29 | 78 | 31 | 90 | 01 | 74 | 31 | 49 | 71 | 48 | 59 | 81 | 16 | 23 | 57 | 05 | 54 |
| 01 | 70 | 54 | 71 | 83 | 51 | 54 | 69 | 16 | 92 | 33 | 48 | 61 | 43 | 52 | 01 | 89 | 71 | 75 | 48 |

What the computer sees

image classification

82% cat  
15% dog  
2% hat  
1% mug

# Convolution Operation

If there was a feature extraction method from images that would follow these principles, we could then train a neural network on the extracted features



# Convolution Operation

- A filter (or kernel) is a small matrix of weights
- The convolution operation: slide the filter across the image, computing a weighted sum (dot product) at each position

$$S(i, j) = \sum_{p=0}^{k-1} \sum_{q=0}^{k-1} \mathbb{X}(i + p, j + q) \cdot \mathbb{K}(p, q)$$

|                 |                 |                 |   |   |
|-----------------|-----------------|-----------------|---|---|
| 1 <sub>x1</sub> | 1 <sub>x0</sub> | 1 <sub>x1</sub> | 0 | 0 |
| 0 <sub>x0</sub> | 1 <sub>x1</sub> | 1 <sub>x0</sub> | 1 | 0 |
| 0 <sub>x1</sub> | 0 <sub>x0</sub> | 1 <sub>x1</sub> | 1 | 1 |
| 0               | 0               | 1               | 1 | 0 |
| 0               | 1               | 1               | 0 | 0 |

Image

|   |  |  |
|---|--|--|
| 4 |  |  |
|   |  |  |
|   |  |  |

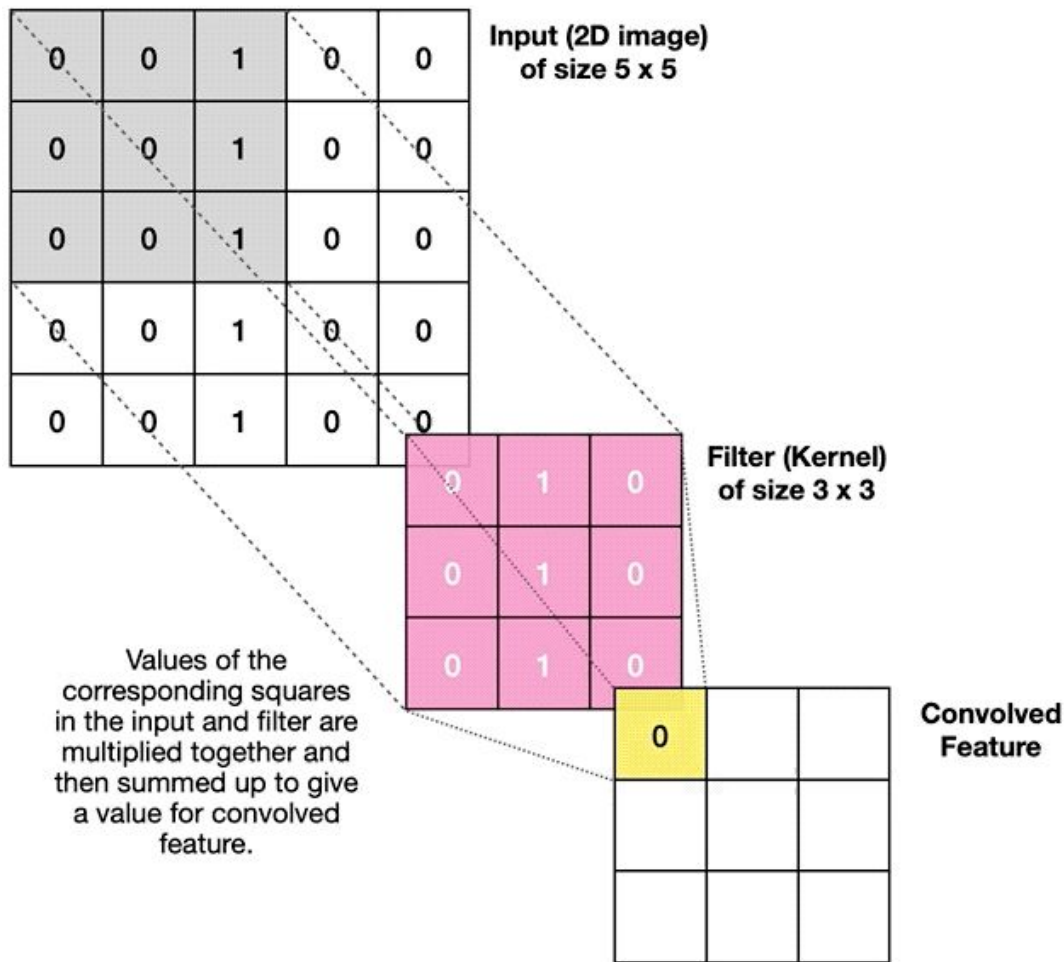
Convolved  
Feature

# Convolution Operation

Why does this matter?

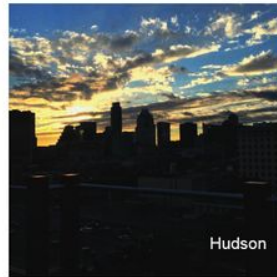
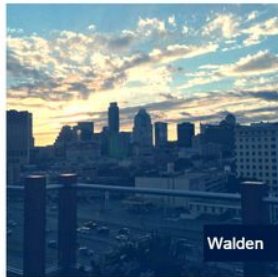
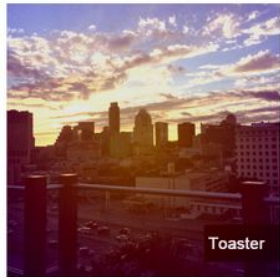
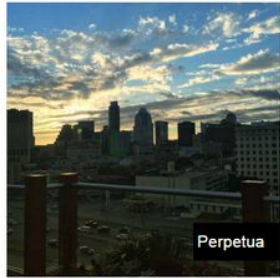
Filters / kernels can extract certain low level features from an image very efficiently

Does this follow translation invariance and locality?





# Convolution Operation



Edge detection

$$\begin{bmatrix} 1 & 0 & -1 \\ 0 & 0 & 0 \\ -1 & 0 & 1 \end{bmatrix}$$



$$\begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$



$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$



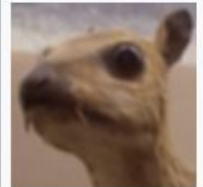
Sharpen

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$



Box blur  
(normalized)

$$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$



# Convolution Operation

Padding:

- Add extra zeros around
- Allows us to use convolution without shrinking the size of the output image too much

What should be the padding if we want to keep the size intact?

|   |     |     |     |     |     |   |
|---|-----|-----|-----|-----|-----|---|
| 0 | 0   | 0   | 0   | 0   | 0   | 0 |
| 0 | 60  | 113 | 56  | 139 | 85  | 0 |
| 0 | 73  | 121 | 54  | 84  | 128 | 0 |
| 0 | 131 | 99  | 70  | 129 | 127 | 0 |
| 0 | 80  | 57  | 115 | 69  | 134 | 0 |
| 0 | 104 | 126 | 123 | 95  | 130 | 0 |
| 0 | 0   | 0   | 0   | 0   | 0   | 0 |

Kernel

|    |    |    |
|----|----|----|
| 0  | -1 | 0  |
| -1 | 5  | -1 |
| 0  | -1 | 0  |

|     |  |  |  |  |
|-----|--|--|--|--|
| 114 |  |  |  |  |
|     |  |  |  |  |
|     |  |  |  |  |
|     |  |  |  |  |
|     |  |  |  |  |

# Convolution Operation

If the original image is  $n \times n$  and the filter is  $f \times f$ , with stride =  $s$  and padding =  $p$ , what is the output size?

$$\left\lfloor \frac{n + 2p - f}{s} + 1 \right\rfloor \times \left\lfloor \frac{n + 2p - f}{s} + 1 \right\rfloor$$

|                |                |                |   |   |   |   |
|----------------|----------------|----------------|---|---|---|---|
| 0 <sub>2</sub> | 0 <sub>0</sub> | 0 <sub>1</sub> | 0 | 0 | 0 | 0 |
| 0 <sub>1</sub> | 2 <sub>0</sub> | 2 <sub>0</sub> | 3 | 3 | 3 | 0 |
| 0 <sub>0</sub> | 0 <sub>1</sub> | 1 <sub>1</sub> | 3 | 0 | 3 | 0 |
| 0              | 2              | 3              | 0 | 1 | 3 | 0 |
| 0              | 3              | 3              | 2 | 1 | 2 | 0 |
| 0              | 3              | 3              | 0 | 2 | 3 | 0 |
| 0              | 0              | 0              | 0 | 0 | 0 | 0 |

|   |    |   |
|---|----|---|
| 1 | 6  | 5 |
| 7 | 10 | 9 |
| 7 | 10 | 8 |

# Convolution vs. Cross-Correlation

True mathematical convolution (as defined in digital signal processing) involves *flipping* the kernel:

$$(\mathbb{X} * \mathbb{K})(i, j) = \sum_{p, q} \mathbb{X}(i - p, j - q) \cdot \mathbb{K}(p, q)$$

Cross-correlation does not flip the kernel:

$$(\mathbb{X} \star \mathbb{K})(i, j) = \sum_{p, q} \mathbb{X}(i + p, j + q) \cdot \mathbb{K}(p, q)$$

# Feature Maps, Channels, and Stacking Filters

For an image with a single channel (black and white images), we only need a single filter, but what about images with multiple channels (colored images)?

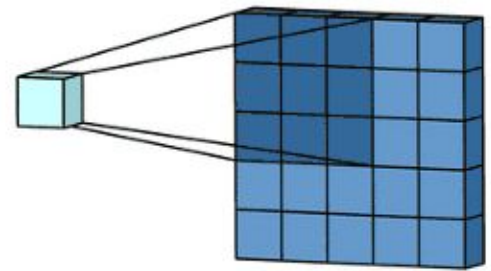
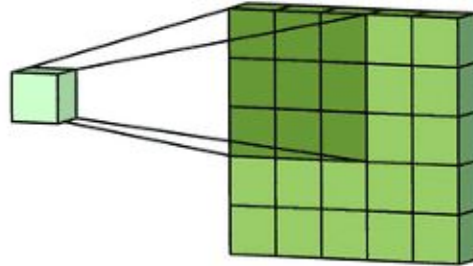
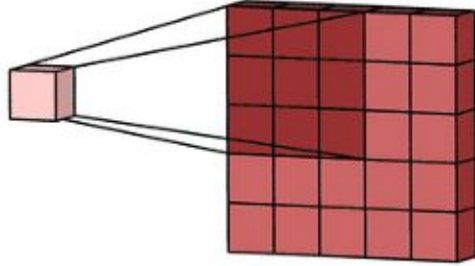
|   |     |     |     |     |     |   |
|---|-----|-----|-----|-----|-----|---|
| 0 | 0   | 0   | 0   | 0   | 0   | 0 |
| 0 | 60  | 113 | 56  | 139 | 85  | 0 |
| 0 | 73  | 121 | 54  | 84  | 128 | 0 |
| 0 | 131 | 99  | 70  | 129 | 127 | 0 |
| 0 | 80  | 57  | 115 | 69  | 134 | 0 |
| 0 | 104 | 126 | 123 | 95  | 130 | 0 |
| 0 | 0   | 0   | 0   | 0   | 0   | 0 |

|        |    |    |
|--------|----|----|
| Kernel |    |    |
| 0      | -1 | 0  |
| -1     | 5  | -1 |
| 0      | -1 | 0  |

|     |  |  |  |  |
|-----|--|--|--|--|
| 114 |  |  |  |  |
|     |  |  |  |  |
|     |  |  |  |  |
|     |  |  |  |  |
|     |  |  |  |  |
|     |  |  |  |  |

# Feature Maps, Channels, and Stacking Filters

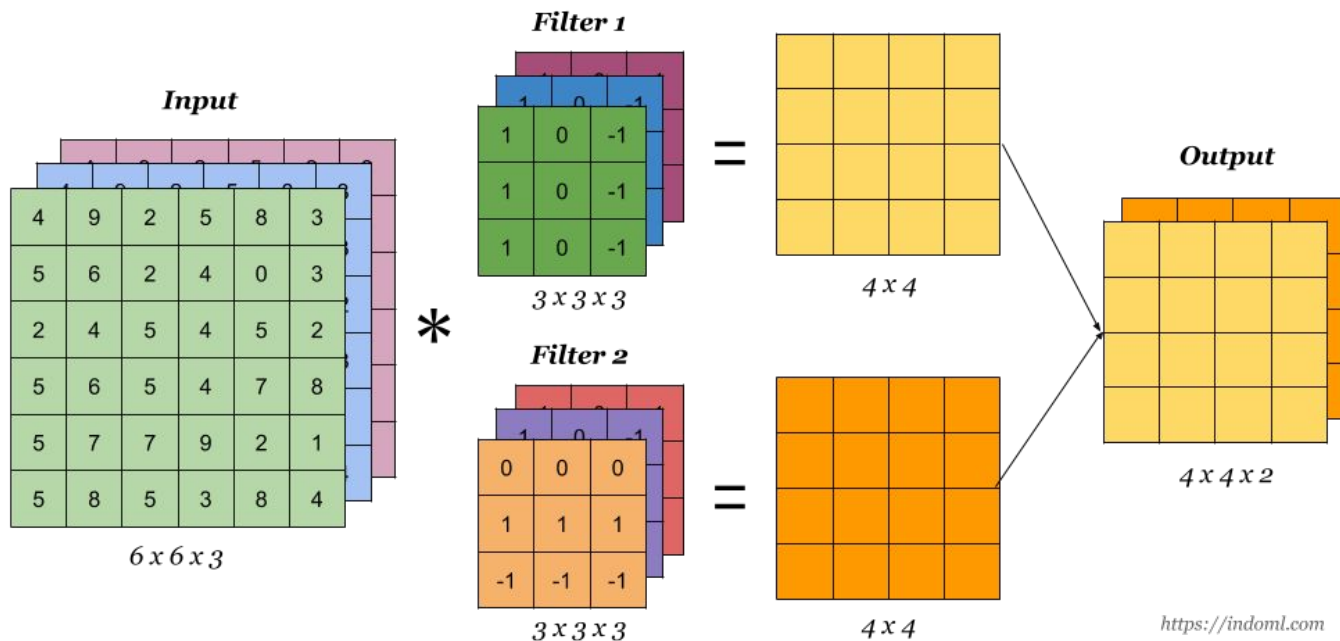
When input has more than one channels, the filter should have matching number of channels





# Feature Maps, Channels, and Stacking Filters

A filter can learn only a single feature, what if you wanted to learn more features?



<https://indoml.com>

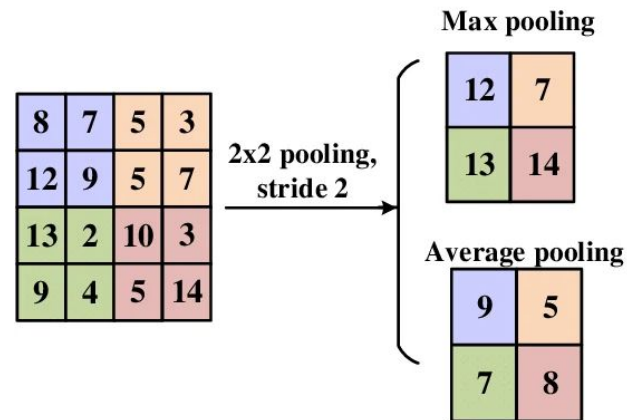
# Pooling Layers

Pooling reduces the spatial dimensions of feature maps by summarising small regions with a single value

- Max Pooling:  $\text{MaxPool}(i, j) = \max_{(p, q) \in \text{window}} \mathbb{X}(i + p, j + q)$
- Average Pooling:  $\text{AvgPool}(i, j) = \frac{1}{p \times q} \sum_{(p, q) \in \text{window}} \mathbb{X}(i + p, j + q)$

**Typical configuration:** 2×2 window with stride 2

Halves both spatial dimensions, reducing total activations by 4 times



# Pooling Layers

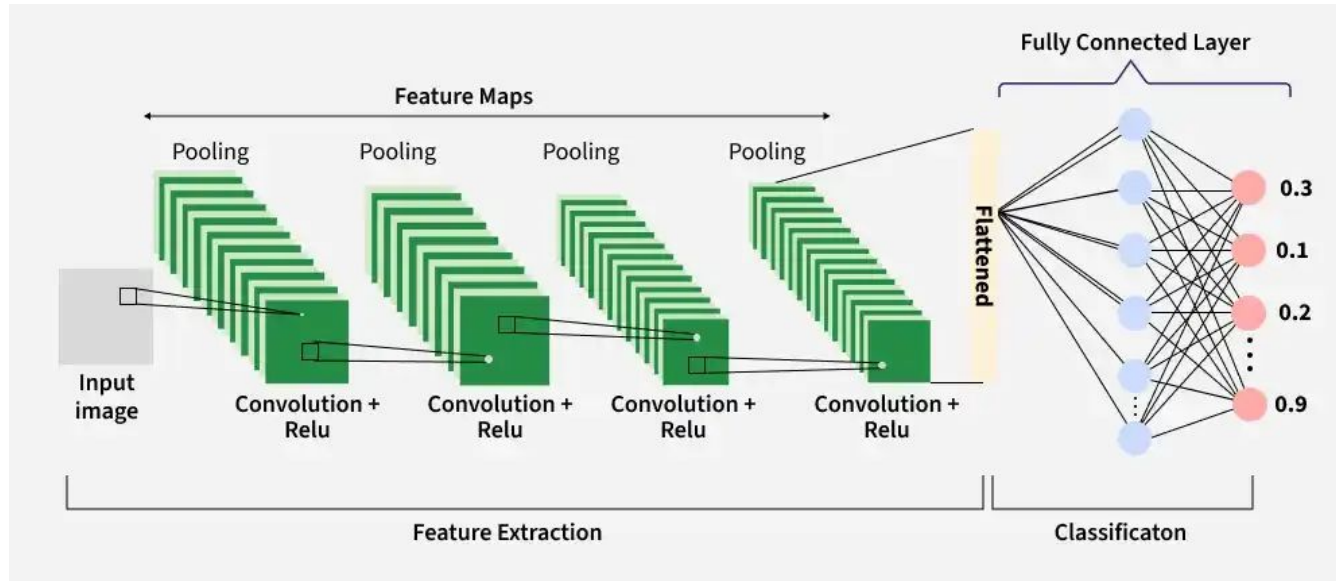
- **Dimensionality reduction:** fewer activations  $\rightarrow$  fewer parameters in subsequent layers  $\rightarrow$  computational efficiency
- **Approximate local translation invariance:** if a feature (e.g., an edge) shifts by some pixels within the pooling window, max-pooling still outputs the same value, making the representation more robust to small spatial shifts
- **Enlarging the effective receptive field:** each subsequent layer "sees" a proportionally larger region of the original image

Many recent architectures replace max pooling with strided convolutions, letting the network learn the downsampling operation rather than fixing it.

# Assembling a Complete CNN

A typical CNN architecture follows a repeating pattern:

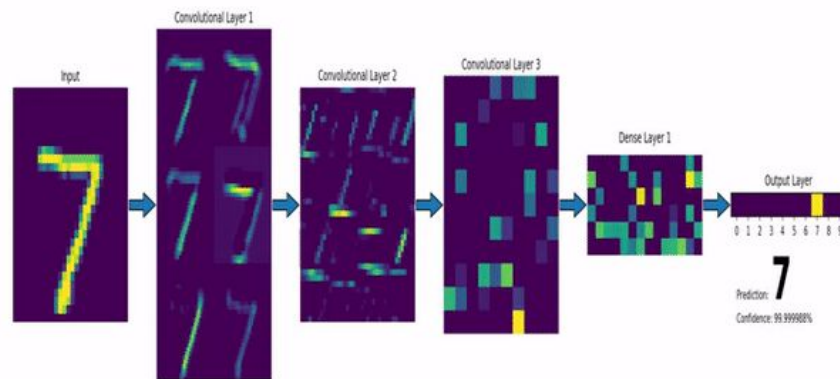
$[\text{Conv} \rightarrow \text{Activation} \rightarrow \text{Pool}] \times N \rightarrow \text{Flatten} \rightarrow \text{Fully-Connected} \rightarrow \text{Output}$



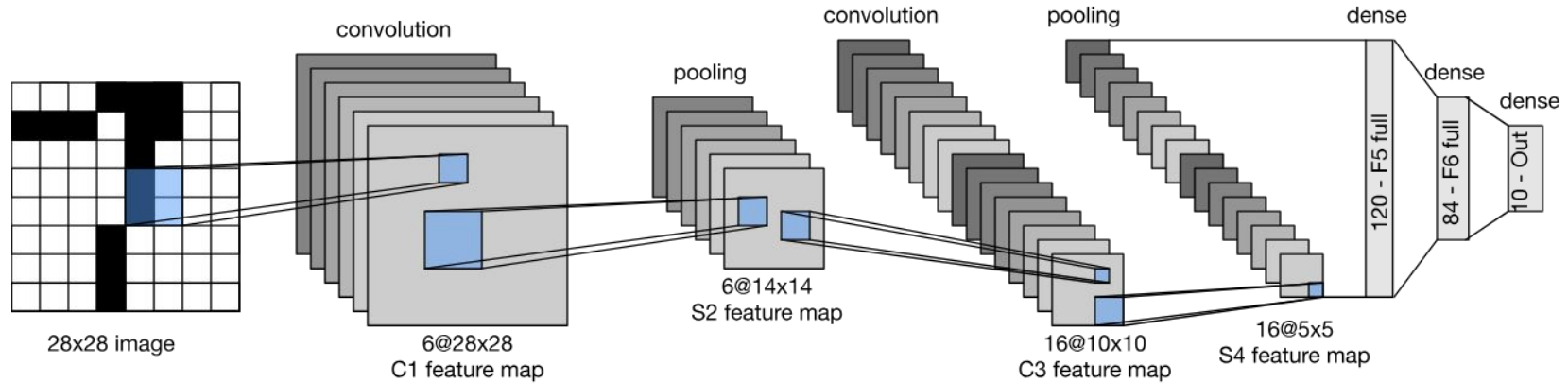
# Assembling a Complete CNN

**Convolutional base** (feature extractor/backbone): stacked conv+pool blocks, progressively reducing spatial size while increasing channel depth

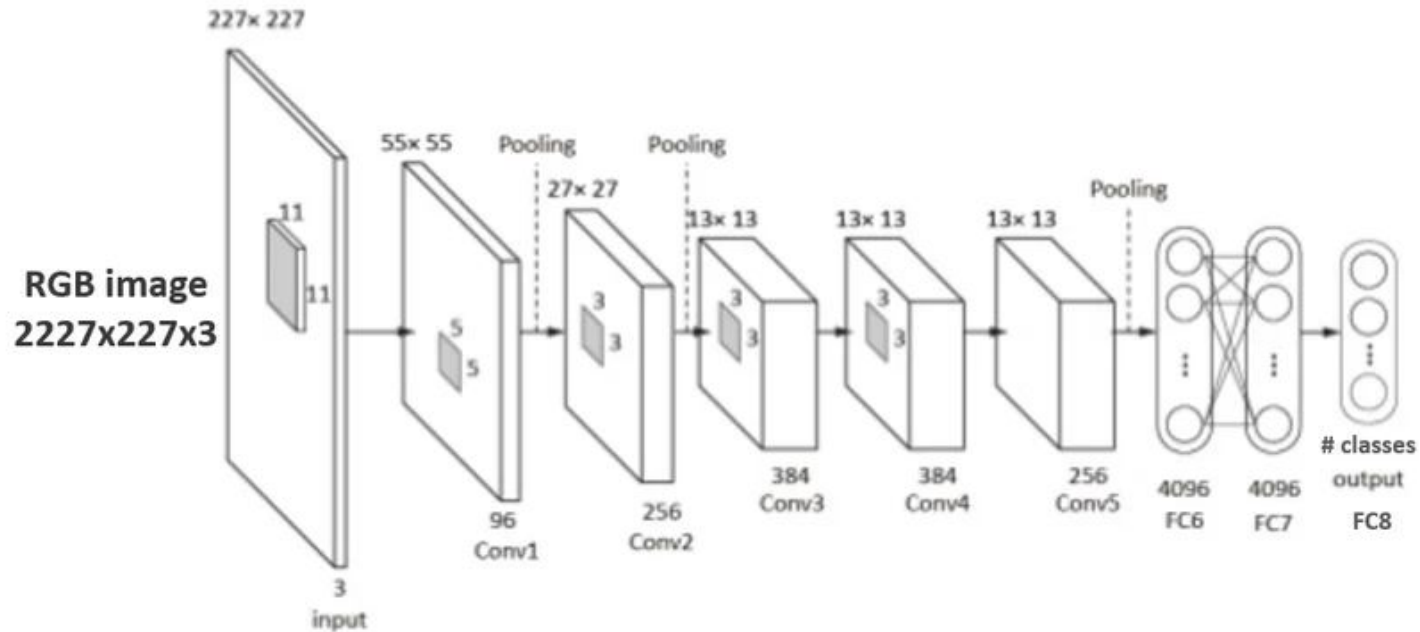
**Classifier head:** flatten the final feature maps, pass through one or more fully-connected layers, ending in an output layer for  $K$  classes



# A Brief Tour of Classic Architectures: LeNet

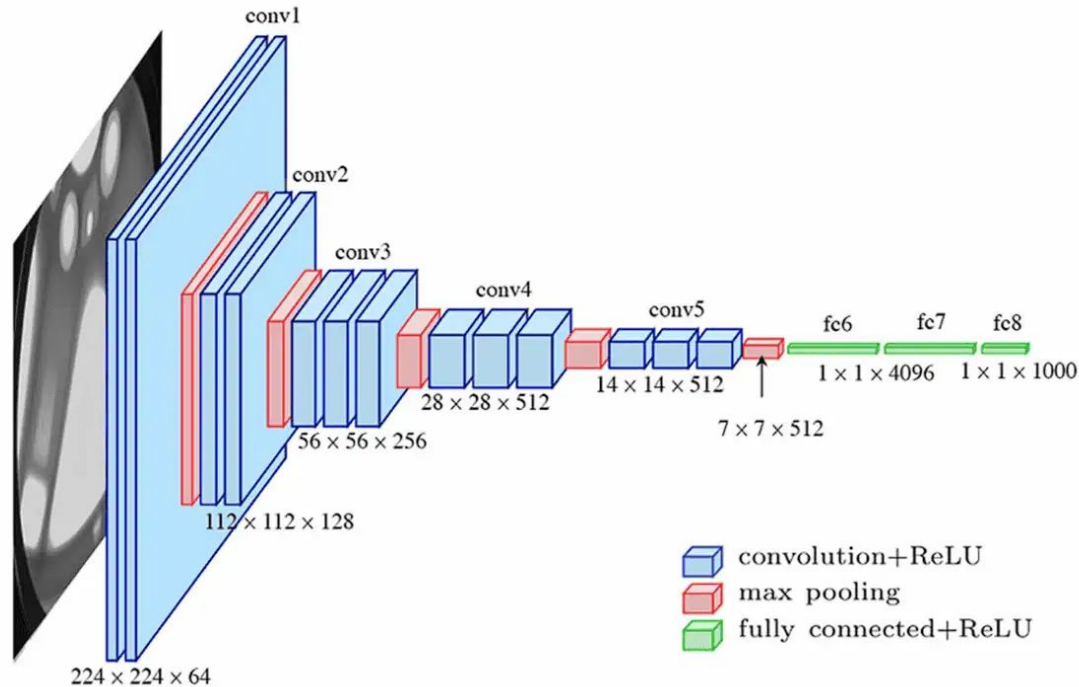


# A Brief Tour of Classic Architectures: AlexNet



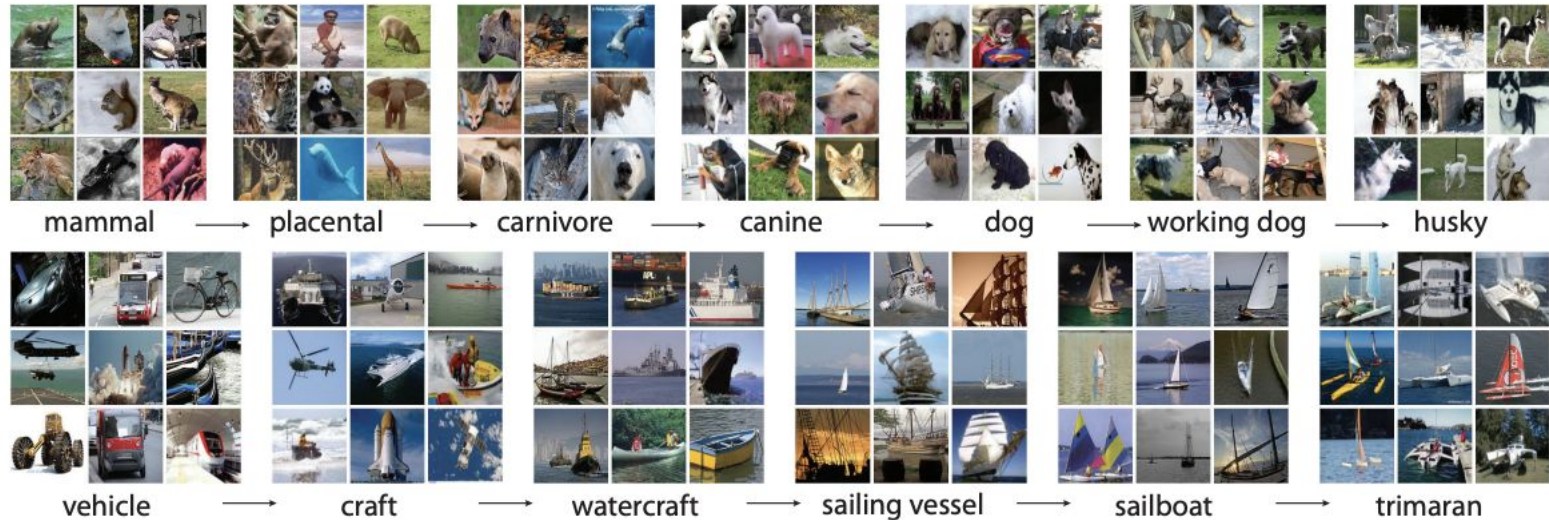


# A Brief Tour of Classic Architectures: VGGNet

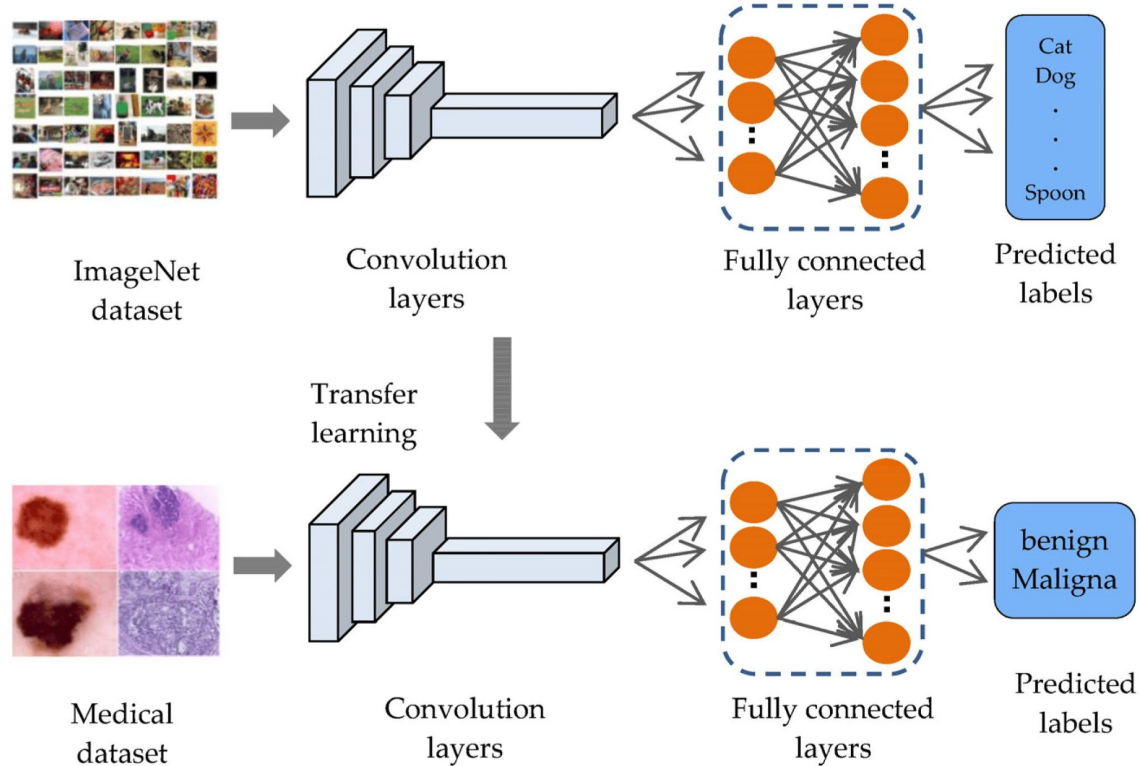


# Transfer Learning

A CNN trained on a large dataset (e.g., ImageNet, 1.2M images, 1000 classes) learns general-purpose visual features in its early/middle layers such as: edges, textures, shapes, object parts. These are useful far beyond the original 1000 classes.

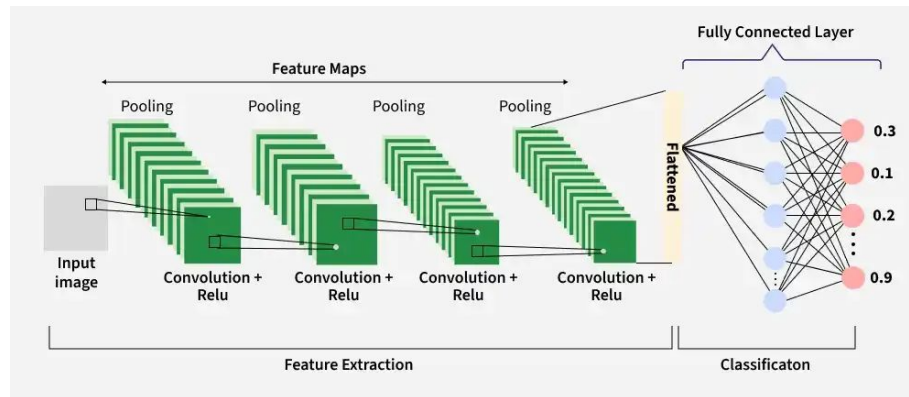


# Transfer Learning



# Transfer Learning

- **Feature extraction** (frozen backbone): Remove the original classifier head, keep the pretrained convolutional base frozen (weights not updated), and train only a new, small classifier head on your new task's data.
- **Fine-tuning**: Unfreeze some or all of the pretrained backbone and continue training (usually with a small learning rate) on the new task, allowing the features to adapt slightly.



# Textbook Chapters

- [Goodfellow] - Chapter 9
- [Alpaydin] - Chapter 12

# Recommended Resources

1. [How we teach computers to understand pictures | Fei Fei Li](#)
2. [CNN Explainer](#)
3. [What are Pooling Layers in Deep Neural Networks?](#)

# Thank you